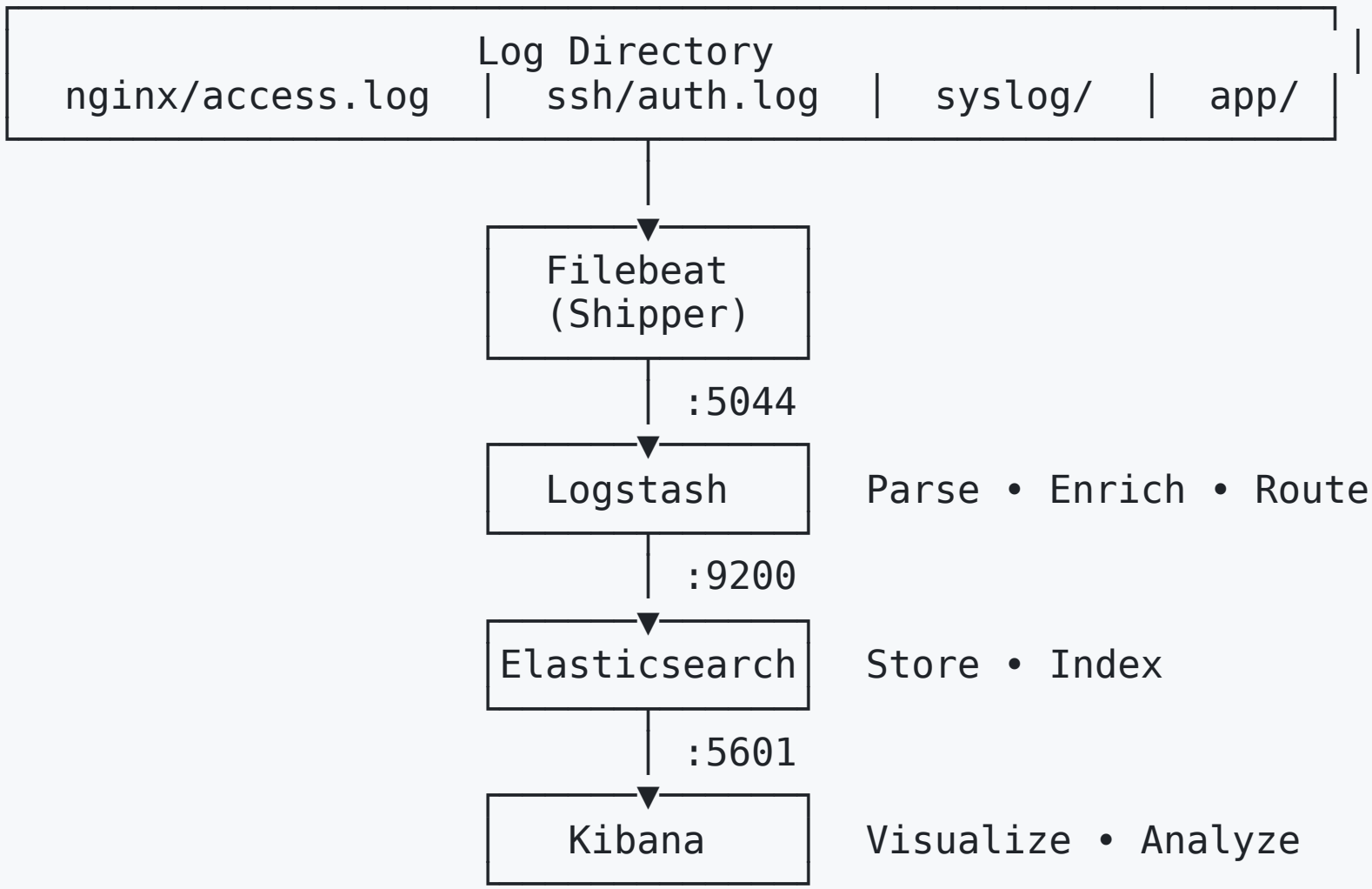
ELK Lab: Observability & Centralized Logging

Systems Reliability Engineering (SRE)

Agenda

- Architecture Overview
- The 5 Log Sources
- Setting Up the Stack
- Configuring Kibana
- Generating Test Data
- Learning Outcomes

Architecture Overview



The 6 Log Sources

Source	Shipper	Format	Key Fields	Use Case
Nginx	Filebeat	Combined Apache Log	clientip, status, request	Web traffic
SSH	Filebeat	JSON	user, result, source_ip, geoip	Security
Syslog (Linux)	Filebeat	RFC5424	hostname, process, message	System events
Syslog (Cisco)	Filebeat	CEF	src_ip, dst_ip, severity	Network logs
App Service	Filebeat	JSON	method, endpoint, latency	App health

Log Source: Windows Event Logs

Shipper: Winlogbeat (not Filebeat)

Format: Windows Event XML

```
Event ID 4624: Account successfully logged on
Event ID 4625: Account failed to log on
Event ID 4634: Logoff
Event ID 4648: Explicit credentials used
```

Key Fields:

- `EventID` → 4624 (success), 4625 (failure)
- `LogonType` → 2 (interactive), 10 (remote), 3 (network)
- `IpAddress` → Source of logon attempt

WEF Architecture: Push vs Pull

Feature	Source-Initiated (Push)	Collector-Initiated (Pull)
Scalability	HIGH - agents send independently	LOW - collector becomes bottleneck
Firewall	Complex - inbound rules needed	Simple - outbound from agents
Configuration	Distributed (per agent)	Centralized (collector config)
Remote Laptops	IDEAL - tolerant of disconnection	Problematic - can't poll offline machines

Recommendation: Source-initiated (Push) for remote/roaming endpoints

Log Source: Nginx

Format: Combined Apache Log

```
192.168.1.50 - - [05/May/2024:14:30:00 +0000] "GET /checkout HTTP/1.1" 200 1234 "-" "Mozilla/5.0"
```

Enriched Fields:

- `clientip` → GeoIP lookup (country, location)
- `timestamp` → Parsed via date filter
- `status` → HTTP status code
- `request` → URL path and method

Log Source: SSH

Format: JSON (structured)

```
{
  "timestamp": "2024-05-05T14:30:00Z",
  "event": "ssh_auth",
  "user": "admin",
  "source_ip": "185.234.72.45",
  "result": "failure"
}
```

Key Insight: 80% successful logins (internal IPs) vs 20% failures (external attack IPs)

Log Source: Syslog (Linux)

Format: RFC5424

```
<34>1 2024-05-05T14:30:00Z webserver ssh 1234 - - Failed password for invalid user admin from 185.234.72.45 port 54321 ssh2
```

Parsed Fields:

- `priority` → severity + facility
- `hostname` → origin system
- `process` → service name
- `message` → the actual event

Log Source: Syslog (Cisco)

Format: CEF (Common Event Format)

```
CEF:0|Cisco|IOS|12.4|5|SSH login attempt|3|src=192.168.1.100 dst=10.0.0.5 spt=54321 dpt=22
```

Fields: Device type, severity level, source/destination IPs, ports

Log Source: App Service

Format: JSON (structured)

```
{
  "timestamp": "2024-05-05T14:30:00Z",
  "method": "POST",
  "endpoint": "/api/checkout",
  "status_code": 200,
  "latency_ms": 145
}
```

Metrics Tracked: Response times, error rates, endpoint popularity

Setting Up: Windows Endpoints

Real Windows Event Forwarding with Winlogbeat

Winlogbeat is the official Beats shipper for Windows Event Logs.

Architecture:

```
Windows Server → Winlogbeat → Logstash :5044 → Elasticsearch
```

Winlogbeat Configuration (winlogbeat.yml):

```
winlogbeat.event_logs:
  - name: Security
    processors:
      - script:
          when.equals.event_id: 4624
          fields:
            logon_result: success
      - script:
          when.equals.event_id: 4625
```

Setting Up: Start the Stack

Step 1: Navigate to docker directory

```
cd elk-lab/docker
```

Step 2: Launch all services

```
docker compose up -d
```

Step 2: Wait for initialization (30-60 seconds)

```
docker compose ps
```

Step 3: Verify services are running

```
curl http://localhost:9200        # Elasticsearch
curl http://localhost:5601/api/status  # Kibana
```

Accessing Kibana

Open in browser:

```
http://localhost:5601
```

First Time Setup:

1. Click "Explore on my own" (skip tutorial)
2. Stack Management → Index Patterns
3. Create patterns for each log source

Configuring Kibana: Index Patterns

Create these patterns in order:

Pattern	Time Field
`logs-nginx-*`	@timestamp
`logs-ssh-*`	@timestamp
`logs-syslog-*`	@timestamp
`logs-cisco-*`	@timestamp
`logs-app-*`	@timestamp
`logs-windows-*`	@timestamp

Note: You can also use `logs-*` to query all sources at once

Configuring Kibana: Create Visualizations

Recommended Visualizations:

1. **SSH Attack Map** (Maps)
    - Layer: Terms on `geoip.location`
    - Source: `logs-ssh-*`
2. **SSH Auth Results** (Pie Chart)
    - Slice by: Terms → `ssh_result`
3. **Nginx Traffic Over Time** (Line Chart)
    - X-axis: Date histogram → `@timestamp`
4. **HTTP Status Codes** (Bar Chart)
    - X-axis: Terms → `status`

Build a Dashboard

Steps:

1. Click **Dashboard** → **Create dashboard**
2. Add all saved visualizations
3. Arrange and resize panels
4. Save as "ELK Lab Overview"

Pro Tip: Use `logs-*` pattern to see all data, or filter by specific source using the `service` field

Generating Test Data

Nginx Logs (web traffic)

```
curl http://localhost:80
curl http://localhost:80/products/123
curl http://localhost:80/checkout
```

SSH Logs (security events)

```
docker exec ssh-simulator /generate_ssh_logs.sh 50
# Generates 80% success (internal) + 20% failure (external)
```

Generating Test Data (continued)

Syslog Events

```
docker exec syslog-simulator /generate_syslog.sh 100
# 50 Linux RFC5424 + 50 Cisco CEF events
```

App Logs (API requests)

```
curl http://localhost:5000/api/health
curl -X POST http://localhost:5000/api/checkout \
  -H "Content-Type: application/json" \
  -d '{"item":"test","user_id":"user-123"}'
```

Verifying Data Arrived

In Kibana Discover:

1. Click **Discover** (left sidebar)
2. Select `logs-*` index pattern
3. Set time range to "Last 15 minutes"
4. You should see documents appearing

Via Command Line:

```
curl localhost:9200/_cat/indices?v
```

Look for indices like: `logs-nginx-2024.05.05`

Logstash Pipeline: How It Works

```
input { beats { port => 5044 } }

filter {
  # Route by service type
  if [service] == "nginx" { grok { ... } geoip { ... } }
  if [service] == "ssh"   { json { ... } geoip { ... } }
  if [service] == "syslog" { grok { ... } }
  if [service] == "cisco"  { grok { ... } }
  if [service] == "app"    { json { ... } }
}

output {
  elasticsearch { index => "logs-%{[service]}-%{+YYYY.MM.dd}" }
}
```

Key: Dynamic index naming by service + date

Learning Outcomes

By completing this lab, you will be able to:

1. **Configure log shippers** - Set up Filebeat to read multiple sources
2. **Parse unstructured logs** - Use Grok filters for field extraction
3. **Enrich telemetry** - Add GeoIP for geographic context
4. **Manage indices** - Create daily indices per log source
5. **Build dashboards** - Visualize correlated telemetry
6. **Analyze security events** - Identify brute-force attacks via SSH + GeoIP
7. **Monitor app health** - Track latency and error rates

Key Commands Reference

```
# Start/stop
docker compose up -d
docker compose down

# View logs
docker compose logs -f filebeat
docker compose logs -f logstash

# Test Elasticsearch
curl localhost:9200/_cat/indices?v

# Generate data
docker exec ssh-simulator /generate_ssh_logs.sh 50
docker exec syslog-simulator /generate_syslog.sh 100

# Restart a service
docker compose restart filebeat
```


Windows Event IDs (Security Log)

Troubleshooting

No data in Kibana?

```
# Check Filebeat
docker exec filebeat filebeat test config
docker logs filebeat

# Check Logstash
docker compose logs logstash | tail -50

# Verify indices exist
curl localhost:9200/_cat/indices?v
```

Services not healthy?

```
docker compose restart
docker compose logs --tail=100
```

Summary

Component	Purpose	Key Config
Filebeat	Ship logs from files	filebeat.yml
Logstash	Parse & enrich	logstash.conf
Elasticsearch	Store & index	docker-compose.yml
Kibana	Visualize	Index patterns

Next Steps: Explore the data, create custom visualizations, build your own dashboard!

Questions?

Lab Repository: `/home/morta/workspace/elk-lab`

Backup: File Structure

```
elk-lab/
├── docker/
│   ├── docker-compose.yml
│   ├── nginx.conf
│   ├── generate_ssh_logs.sh
│   ├── generate_syslog.sh
│   └── app/
├── config/
│   ├── filebeat.yml
│   └── logstash.conf
├── logs/
│   ├── nginx/
│   ├── ssh/
│   ├── syslog/
│   └── app/
└── docs/
    └── README.md
```